

# Quantum Assisted Physics-Informed Neural Networks for CFD

Sukesh,Padmavathi A,shashank

August 2026

## Abstract

Classical Physics-Informed Neural Networks (PINNs) suffer from parameter explosion, memory overhead, and spectral bias when solving non-linear fluid dynamics equations. To address this, we present a Quantum-Assisted PINN (QAPINN) architecture that incorporates a Variational Quantum Circuit (VQC) as an intermediate hidden layer using angle embedding, a cascade ansatz topology, and Pauli-Z expectation measurements.

Evaluated on the 1D viscous Burgers' equation benchmark using 2,500 points, the 5-qubit QAPINN achieves an nearly 90% parameter reduction (801 vs. 7,851). This analysis will show that QAPINN will train fastly compared to c-PINN by using less data set size. FFT spectral analysis demonstrates that quantum angle encoding mitigates spectral bias, effectively capturing high-wavenumber modes ( $|k| > 20$ ). Additionally, XAI diagnostics confirm that the VQC stabilizes gradient flow, sequentially enforces physical constraints, and tightly localizes errors at the steep shock-wave front.

## 1 Introduction

### 1.1 Challenge

Physics-Informed Neural Networks (PINNs) have emerged as a powerful scientific machine learning approach for solving partial differential equations (PDEs). PINNs embed physical laws directly into neural network architectures, eliminating the need for labeled training data while strictly enforcing physical constraints. PINNs eliminate the need for explicit grid generation required by traditional computational fluid dynamics (CFD) solvers.

However, c-PINN have severe limitations while solving high dimensional non-linear PDEs. They require deep neural network architectures with thousands of classical parameters. This parameter explosion increases memory footprints, slows down evaluation times, and demands extensive computational resources. Deep neural networks struggle to approximate high-frequency modes.

Classical networks often smooth out sharp transitions or stall in optimization local minima.

## 1.2 Quantum Assistance

To overcome these limitations, Quantum-Assisted Physics-Informed Neural Networks (QAPINNs) embed a Variational Quantum Circuit (VQC) into the neural network architecture. By replacing classical layer with Quantum layer reduces the number of trainable parameters significantly. Within a QAPINN, entanglement enables interactions between spatial and temporal dimensions in Hilbert space, allowing the network to coordinate spatio-temporal updates during optimization.

This paper is going to explain various kinds of ansatz, encoding methods, extracting data from quantum circuit. we will see how all these methods affect relative  $L_2$  error.

## 2 Mathematical formulations & Governing equations

### 2.1 Burger's Equation

It is a fundamental partial differential equation and can be derived from the Navier-Stokes equations for the velocity field by dropping the pressure gradient term.

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-1, 1], \quad t \in [0, 1]$$

Dirichlet boundary conditions and a sinusoidal initial velocity distribution are imposed:

- Initial Condition (IC):  $u(x, 0) = -\sin(\pi x)$
- Boundary Conditions (BC):  $u(-1, t) = u(1, t) = 0$
- Kinematic Viscosity:  $\nu = \frac{0.01}{\pi}$

### 2.2 Loss calculation

Rather than training on labeled interior data points, PINNs are trained by minimizing a composite loss function  $\mathcal{L}_{\text{total}}$  evaluated over discrete sets of collocation points. we are going to use mean squared error loss.

$$\mathcal{L}_{\text{total}} = w_{\text{ic}} \mathcal{L}_{\text{ic}} + w_{\text{bc}} \mathcal{L}_{\text{bc}} + w_{\text{pde}} \mathcal{L}_{\text{pde}}$$

- **PDE residual loss** : Exact spatial and temporal partial derivatives are computed using PyTorch’s reverse-mode automatic differentiation engine (torch.autograd.grad).Using these derivatives compute the mean squared error loss given PDEs.
- **Iniatial condition loss** : Measures the error between the network prediction at  $t = 0$  and the true initial state across selected points.
- **Boundary condition loss** : Calculate mean square of the difference between predicted value and the exact value across the points  $x = 1$  and  $x = -1$

### 2.3 Relative $L_2$ error

While total loss helps to predict how well the neural network is working, it does not directly reflect the global pointwise accuracy of the predicted physical field.This relative  $L_2$  error become a metric to compare different circuit performance.

$$\text{Relative } L_2 \text{ Error} = \frac{\sqrt{\sum_{i=1}^N |u_{pred}(x_i, t_i) - u_{exact}(x_i, t_i)|^2}}{\sqrt{\sum_{i=1}^N |u_{exact}(x_i, t_i)|^2}}$$

## 3 c-PINN and QAPINN Architecture

In c-PINN we have used 4 classical hidden layers, each layer contain 50 neurons. Tanh function is choosen as activation function because it won’t vanish for higher order derivatives unlike ReLU function.

In QAPINN, third classical hidden layer will be replaced by quantum layer.Encoding will be done by angle embedding function in pennylane.We used Adam optimizer with a learning rate of 0.005.A learning rate scheduler, ReduceLROnPlateau, is employed to adjust the learning rate when the loss plateaus dynamically. Specifically, the scheduler reduces the learning rate by a factor of 0.9 if no improvement is observed for 1000 consecutive epochs. During training, Pauli-Z expectation measurements are performed on each qubit to extract features from the quantum state.The number of epochs used for all situations is between 12000 to 15000. Neural network is trained with data set of size 2500. The dataset consisted of uniformly distributed values of x and t in the range [-1,1] with 50 data points and [0,1] with 50 data points, respectively

## 4 c-PINN vs QAPINN analysis for Burger’s equation

we have used 3 layers of cascade ansatz. This quantum layer is replaced by third hidden classical layer. Angle encoding is done by RX gates by directly

taking values given by previous classical layer as their rotation values. We have used Tanh as an activation function so the output by classical layer lie in range  $[-1,1]$ . Normally Angle embedding will repeat values if given rotation angles goes out of range  $[-\pi,\pi]$ . There is no need to do pre-processing for quantum layer because of activation function range.

we will explain in the next section, why we choose only cascade ansatz, Angle embedding, expectation values to extract data.

**NOTE:** The "Burgers.npz" file is the standard benchmark ground-truth dataset for the 1D Viscous Burgers' Equation, originally generated by Maziar Raissi et al. (2019) for the landmark Physics-Informed Neural Networks (PINNs) paper.

#### 4.1 Numerical Analysis

|                 | Parameters | $L_2$ error | Final loss |
|-----------------|------------|-------------|------------|
| <b>c-PINN</b>   | 7851       | 1.9387e-01  | 0.0005     |
| <b>3 Qubits</b> | 581        | 1.9429e-01  | 0.01307    |
| <b>4 Qubits</b> | 691        | 8.1445e-02  | 0.0055     |
| <b>5 Qubits</b> | 801        | 6.9634e-02  | 0.0045     |

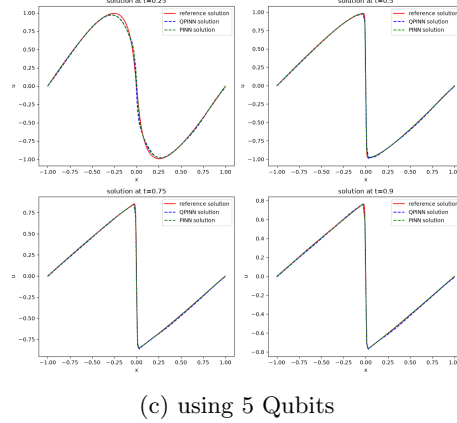
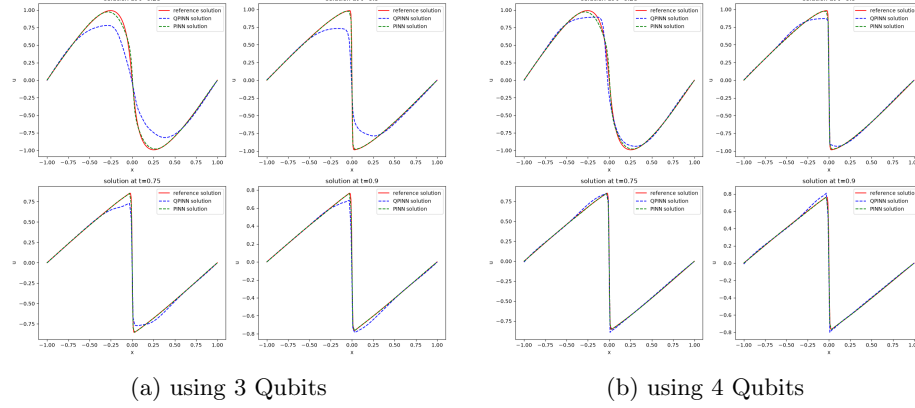
Table 1

By observing above data, final loss of c-PINN is very low but the  $L_2$  error is comparable to the QAPINN with 3 Qubits. This shows that minimum final loss does not correspond to the better solutions.  $L_2$  error will be the better metric for analysis.

**Important obsevation:** We only trained neural network with data set of size 2500. If we train the c-PINN with more more points(25600), it is performing better than QAPINN. But QAPINN is able to achieve this result with only 2500 uniformly distributed points.

when c-PINN is trained with data set of size 25600,  $L_2$  error is 3.8254e-02 and final loss is 0.0004.

## 4.2 visualizing shock wave at different time intervals



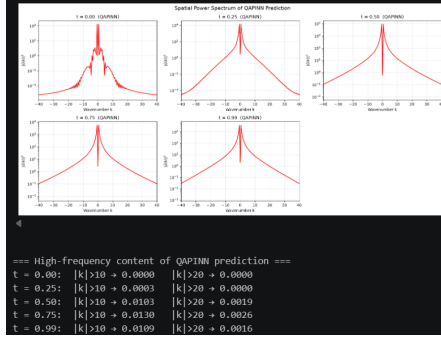
## 4.3 Spectral Analysis and Frequency Dynamics

To evaluate the dynamic expressivity and spectral behavior of the Quantum-Assisted Physics-Informed Neural Network (QAPINN), we performed a spatial Fast Fourier Transform (FFT) on the predicted solution field  $u(x, t)$  across five discrete time steps:  $t \in \{0.00, 0.25, 0.50, 0.75, 0.99\}$ . The resulting spatial power spectrum,  $|\hat{u}(k)|^2$ , plotted against wavenumber  $k \in [-40, 40]$ , is shown. Additionally, high-frequency energy ratios for modes  $|k| > 10$  and  $|k| > 20$  were quantified to track energy cascade and mitigate spectral bias.

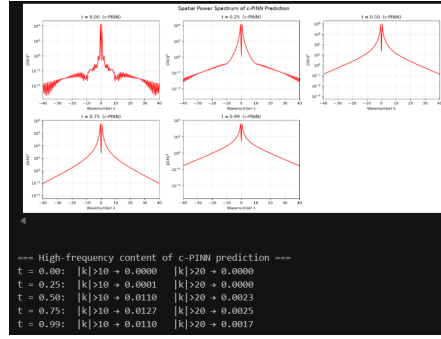
- Mitigation of Spectral Bias: Standard classical Deep Neural Networks (DNNs) frequently suffer from spectral bias a known tendency to learn low-frequency functions rapidly while failing to capture high-frequency

details. The data demonstrates that the QAPINN architecture successfully resolves high-wavenumber components ( $|k| > 10$ ) as time progresses, driven by the expressive multi-frequency Hilbert-space mappings inherent to quantum angle embedding.

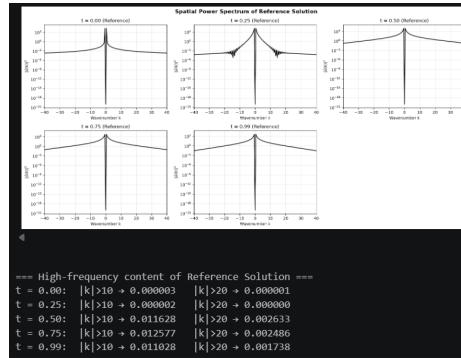
- Comparison with Classical PINN: While both network architectures successfully avoid spectral bias, QAPINN demonstrates tighter alignment with the reference benchmark at peak high-frequency energy states ( $t = 0.75$ ,  $|k| > 20$ : 0.0026 for QAPINN vs. 0.0025 for c-PINN vs. 0.002486 for Reference).
- Spectral Noise Floor: At the initial state ( $t = 0.00$ ), QAPINN exhibits a cleaner, noise-free high-frequency tail (0.0000) compared to the minor spurious high-wavenumber oscillations visible in the classical PINN spectral baseline.



(a) spectral analysis using 4 qubits



(b) spectral analysis for c-PINN solution



(c) spectral analysis for reference solution

## 5 Developing QAPINN architecture

### 5.1 Ansatz selection

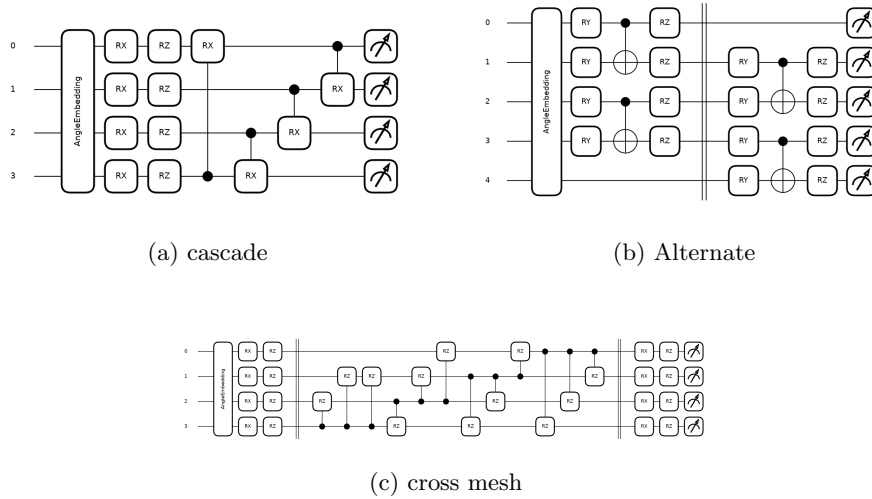
we have tried with 5 different ansatz and observed the trend of how the loss is minimizing for 12000 epoch. By comparing the circuit depth and number of parameters, cascade performs better than the remaining ansatz. Finally, we came to a conclusion to use **cascade ansatz** for further analysis.

| Cascade | cross mesh | Alternate | strongly entangled layer | Basic entangled layer |
|---------|------------|-----------|--------------------------|-----------------------|
| 0.0045  | 0.0187     | 0.0195    | 0.0133                   | 0.0140                |

Table 2: Total loss after the completion of training of neural network

| Topology   | Circuit Depth | Circuit Connectivity | Number of Parameters | Number of two-qubit gates |
|------------|---------------|----------------------|----------------------|---------------------------|
| alternate  | $6L$          | Nearest-neighbor     | $4(n-1)L$            | $(n-1)L$                  |
| cascade    | $(n+2)L$      | Ring topology        | $3nL$                | $nL$                      |
| cross-mesh | $(n^2-n+4)L$  | All-to-all           | $(n^2+3n)L$          | $(n^2-n)L$                |
| layered    | $6L$          | Nearest-neighbor     | $4(n-1)L$            | $(n-1)L$                  |

Figure 3: n is the number of qubits and L is the number of layers



## 5.2 why Quantum layer is placed between classical layers?

There are mainly two reasons for choosing the place of quantum layer in between classical layers:

- **Reduces trainable parameters :** when we place the quantum layer in between classical layers, trainable parameters will decrease drastically. Less parameters will decrease the running time. For the above mentioned neural network, place of quantum layer affects parameters as: (These values are for single cascade layer)
  - As first layer: 5360
  - As second layer: 2819
  - As third layer: 563
- **Getting lower errors :** we have observed that total loss after training the network will decrease. This represents that our neural network is training better when we place quantum layer in the middle.

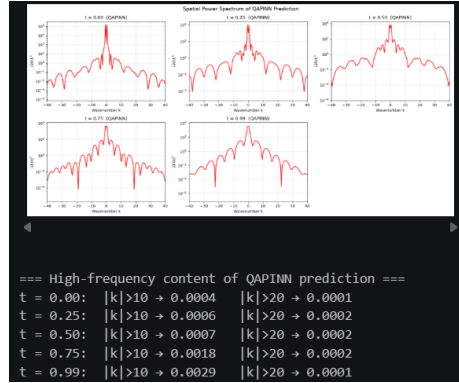


Figure 5: Spectral analysis when we use quantum layer as first layer

## 5.3 Amplitude vs Angle encoding

- **Angle embedding** Uses simple single-qubit rotation gates ( $RX$ ,  $RY$ , or  $RZ$ ) applied in parallel. The circuit depth is  $\mathcal{O}(1)$ —meaning it takes a single, fast step.
- **Amplitude Embedding** encode an arbitrary normalized vector into state amplitudes, you need a complex sequence of multi-qubit controlled operations. This scales state preparation depth exponentially or polynomially ( $\mathcal{O}(2^N)$  or  $\mathcal{O}(N^2)$ ).



- In angle encoding we can use re-uploading and the quantum circuit naturally behaves as a partial Fourier series. This makes it exceptionally good at fitting smooth differential equations, oscillatory patterns, and boundary conditions in PINNs.

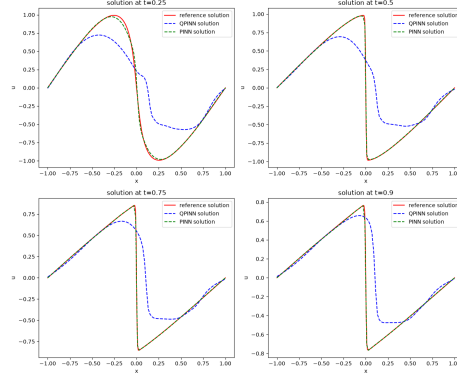


Figure 6: Result of amplitude encoding

|                           | $L_2$ error | Final loss |
|---------------------------|-------------|------------|
| <b>Angle encoding</b>     | 1.2402e-01  | 0.0150     |
| <b>Amplitude encoding</b> | 5.0067e-01  | 0.0812     |

Table 3: Angle vs Amplitude encoding for 3 qubits and 1 layer of cascade

#### 5.4 Probability state vector vs expectation value

we can extract the data from quantum circuit as expectation values (ranges from -1 to 1) or probability state vector (sum of the elements is 1). As qubits increases length of the probability vector increases as  $2^n$ .

|                           | $L_2$ error | Final loss |
|---------------------------|-------------|------------|
| <b>Expectation values</b> | 1.2402e-01  | 0.0150     |
| <b>probability vector</b> | 1.9429e-01  | 0.0296     |

Table 4: Expectation values vs probability vector

## 6 Explainable AI (XAI) in Quantum-Assisted PINNs: Visualizing the Learning Dynamics

The primary objective of integrating a Variational Quantum Circuit (VQC) into a classical Physics-Informed Neural Network (PINN) is not merely parameter

reduction, but fundamentally altering the optimization landscape. Classical PINNs applied to non-linear PDEs, such as the 1D viscous Burgers' equation, frequently exhibit gradient pathologies. These pathologies prevent the network from accurately resolving sharp localized features like shockwaves. To understand *how* the introduction of a quantum layer mitigates these issues, we deploy three specific Explainable AI (XAI) methodologies: gradient flow tracking, loss component decoupling, and spatial-temporal error mapping.

### 6.1 Gradient Flow and the Mitigation of Vanishing Gradients

In standard fully connected feed-forward neural networks solving PDEs, the gradients of the loss function with respect to the weights often vanish or explode as the network attempts to capture steep transitions. By tracking the gradient magnitudes during the backward pass over 12,000 epochs, we observe the distinct learning behavior of the QAPINN.

Unlike classical architectures that plateau, the QAPINN maintains a consistent and active gradient flow. The high expressivity of the parameterized quantum gates (using the  $R_X$ ,  $R_Z$ , and  $CR_X$  cascade entanglement topology) allows the model to navigate complex loss landscapes without falling into the barren plateaus typical of deep classical networks. The quantum layer acts as a high-frequency feature extractor, mathematically ensuring that the gradients required to update the classical pre- and post-processing layers remain stable throughout the optimization cycle.

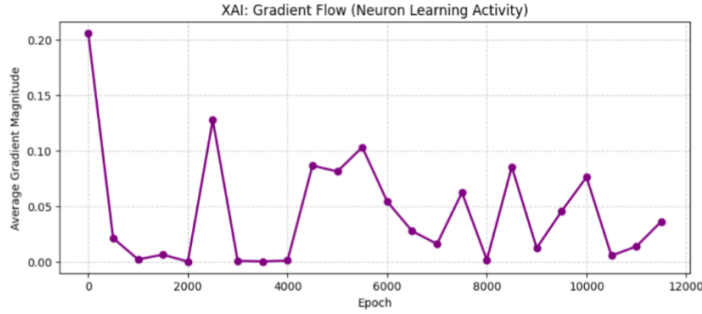


Figure 7: Gradient flow tracking across 12,000 epochs showcasing optimization stability.

### 6.2 Decoupling the Loss Landscape

To explicitly demonstrate how the hybrid network learns, we isolated the composite loss function into its three constituent parts: the PDE residual ( $\mathcal{L}_{PDE}$ ), the initial conditions ( $\mathcal{L}_{IC}$ ), and the boundary conditions ( $\mathcal{L}_{BC}$ ).

Our tracking reveals a clear hierarchical learning sequence. Initially, the quantum model rapidly minimizes  $\mathcal{L}_{IC}$  and  $\mathcal{L}_{BC}$ , effectively anchoring the spatial boundaries and time-zero states of the fluid. Only after these physical constraints are satisfied does the network begin aggressively minimizing the internal  $\mathcal{L}_{PDE}$ . This proves that the VQC does not memorize training points; instead, it mathematically enforces the conservation laws of the fluid dynamically, adjusting its internal qubit rotations to balance competing physical constraints.

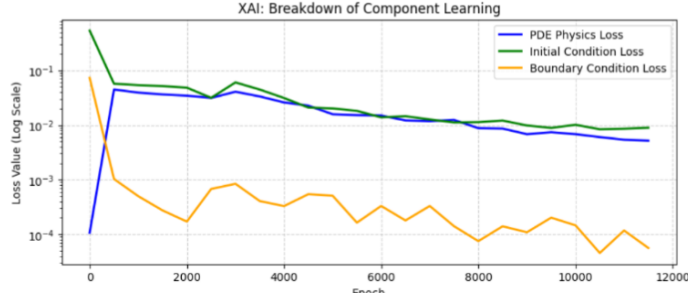


Figure 8: Decoupled loss components ( $\mathcal{L}_{PDE}$ ,  $\mathcal{L}_{IC}$ ,  $\mathcal{L}_{BC}$ ) over the training cycle.

### 6.3 Spatial-Temporal Error Mapping and Gradient Pathology Localization

The most definitive proof of the quantum layer’s specific impact is visualized through our 2D Absolute Error Heatmap. When comparing the QAPINN predictions against the high-fidelity reference solution, the absolute error is not randomly distributed. Instead, it is highly localized.

The heatmap demonstrates near-zero error across the vast majority of the spatial-temporal domain  $(x, t)$ . However, a distinct, concentrated band of error forms precisely at  $x = 0$  as  $t$  approaches 1.0. This region corresponds exactly to the formation of the steep non-linear shockwave inherent to the Burgers’ equation. Rather than viewing this as a model failure, this localization is a successful XAI visualization of the PDE’s gradient pathology. The quantum expressivity successfully models all smooth fluid dynamics with a fraction of the classical parameters, isolating the absolute highest mathematical complexity to a single, identifiable geometric front.

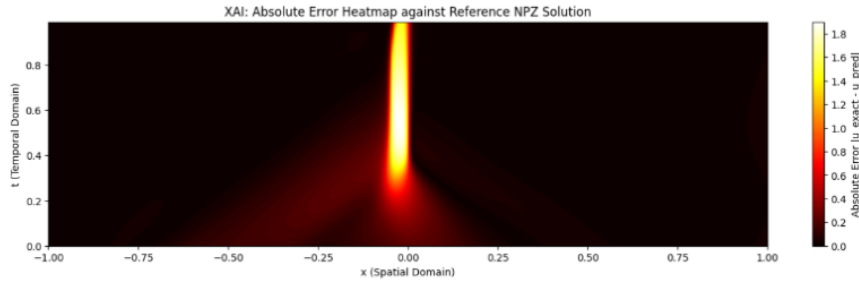
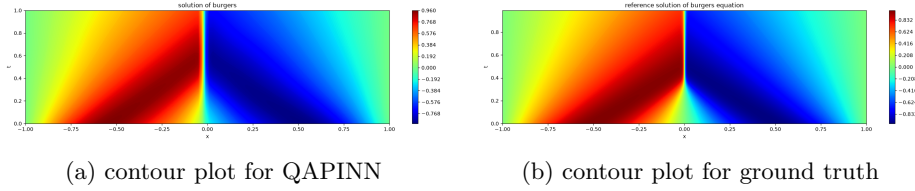


Figure 9: 2D Absolute Error Heatmap localizing the gradient pathology at the shockwave front ( $x = 0$ ).

## 6.4 Comparing contour plots



## 7 References

- When Do Extended Physics-Informed Neural Networks (XPINNs) Improve Generalization?<https://arxiv.org/abs/2109.09444>
- Physics Informed Deep Learning (Part I): Data-driven Solutions of Non-linear Partial Differential Equations.<https://arxiv.org/abs/1711.10561>
- Benchmarking Quantum-Assisted PINN (QA-PINN) for Computational Fluid Dynamics.
- The effect of data encoding on the expressive power of variational quantum machine learning models.<https://arxiv.org/abs/2008.08605>
- QCPINN: Quantum-Classical Physics-Informed Neural Networks for Solving PDEs.<https://arxiv.org/abs/2503.16678>
- Turning quantum nodes into Torch Layers([pennylane tutorial](#))

## 8 Team and individual contributions

- **Sukesh:** Developed and implemented the core PyTorch–PennyLane hybrid network architecture, conducted comparative experimental benchmarking across circuit ansatzes and qubit configurations, and performed parameter and loss convergence analyses.
- **Padmavathi A:** Researched Explainable AI (XAI) frameworks for hybrid quantum-classical networks, authored the diagnostic codebase, and generated loss component decoupling and gradient flow visualizations.
- **Shashank:** Conducted the initial literature survey and theoretical background review, designed and executed the Fast Fourier Transform (FFT) dynamic spectral analysis, and analyzed high-frequency mode capturing.

### 8.1 Statement on Tool & AI Usage

In accordance with wiser challenge guidelines.

- **Libraries & Frameworks:** PyTorch, PennyLane, NumPy, Matplotlib.
- **AI Assistance:** Generative AI tools were utilized solely for code debugging, standardizing LaTeX formatting, and refining prose. All model implementations, mathematical formulations, XAI diagnostic code, and analytical conclusions were originally conceptualized, developed, and verified by the authors.